Q1

Which function is fastest for creating a large array that you will immediately overwrite?

Np.empty()

Q2

Why is `np.arange(0, 1, 0.1)` sometimes unsafe?

May produce precision issues due to floating-point representation.

Q3

What is the core difference between `arange()` and `linspace()`?

Arrange() provide elements that are evenly spaced within interval and linspace() provide elements that are evenly spaced over interval.

Arrange() uses steps, linspace() uses number of points.

Q4

If you don't specify dtype while creating an array of integers, what dtype does NumPy usually choose on 64-bit systems?

Numpy will recognize the elements in the array where they are int or float and then assigned a data type that could be accepted by all elements.

Q5

Which function would you use to create a 5×5 identity matrix?

Np.eye(5)

Q6

Why is np.full() slightly slower than np.zeros()?

np.zeros() may use optimized memory clearing (memset),
while np.full() must assign value element by element.

Q7

What is the memory formula when creating any array?

Nbytes = size * itemsize

Q8

Why is default_rng() preferred over older np.random functions?

Reproducibility, performance.

Q9

If you create np.zeros((1000,1000), dtype=np.float32) instead of float64, what is the main advantage?

Memory efficiency and no memory wastage, future computation speed, performance.

Q10

Which function should you use if you need exactly 50 evenly spaced float numbers between 0 and 10?

Np.linspace(0,10,50)